Technical Specification: Microcircuit Assembly

Spatially Embedded Adjacency Matrix Generation

Research and Development Documentation

March 26, 2026

1 Introduction

This document outlines the core logic of the microcircuit assembly system. The primary objective is the generation of a biologically grounded **Spatially Embedded Adjacency Matrix**. Unlike traditional models that assume random connectivity (Erdős–Rényi graphs), this implementation leverages 3D spatial relationships and empirical data from the *Markram et al. (2015) S1 model* to determine neuronal connectivity.

2 Core Functionality: `build_adjacency_matrix`

The function constructs an $N \times N$ binary directed graph where each entry A_{ij} represents the presence (1) or absence (0) of a synaptic connection from neuron i to neuron j.

2.1 1. Spatial Relationship Modeling

The function begins by computing the 3D Euclidean distance between every pair of neurons in the column:

$$d(i,j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2} \tag{1}$$

To ensure high performance for large populations (e.g., $N > 30,000$), it utilizes `scipy.spatial.distance.`. This vectorized approach calculates all N^2 distances in optimized C-code, avoiding the $O(N^2)$ overhead of standard Python loops.

2.2 2. Pathway-Specific Probability Calculation

Biological connectivity is non-uniform across cell types. The function iterates through unique morphological type (m-type) pathways (e.g., $L23\_PC \rightarrow L4\_SS$) and selects the most accurate mathematical model for that specific projection based on `conn_data`.

A. Analytical Fits (Exponential & Gaussian)

If a pathway has been analytically modeled, the function applies one of two decay formulas:

- **Exponential Decay (exp):** Used for pathways that are densest at close range and drop off steadily.

$$P(d) = A_0 \cdot \exp\left(-\frac{\max(d - d_0, 0)}{\lambda}\right) \tag{2}$$

  Where A_0 is the peak probability, d_0 is the plateau distance, and λ is the length constant.

- **Gaussian Decay (gauss):** Typically used for inhibitory interneurons that cluster symmetrically around their targets.

$$P(d) = A_0 \cdot \exp\left(-\frac{(d - x_0)^2}{\lambda^2}\right) \tag{3}$$

B. Empirical Step-Function (Binned Fallback)

For pathways without a smooth analytical fit, the function uses a binned fallback approach, mapping distances to 16 discrete empirical bins (ranging from $12.5\,\mu$m to $375\,\mu$m).

- **Distance Masking:** A boolean mask is applied to the distance matrix to assign probabilities found in empirical matrices (e.g., `pmat12um`, `pmat25um`).
- **Spatial Cutoff:** Distances exceeding the empirical maximum ($375\,\mu$m) are automatically assigned a probability of 0.0, reflecting the physical limits of axonal reach.

2.3 3. Biological Constraints & Stochastic Sampling

Once the continuous probability matrix (`prob_matrix`) is populated, two final steps generate the binary graph:

1. **Autapse Prevention:** Biological neurons rarely synapse onto themselves. The function zeroing out the diagonal: $A_{ii} = 0$.
2. **Bernoulli Trial:** A Monte Carlo sampling is performed. A matrix of uniform random numbers $R \in [0, 1)$ is generated. A connection is formed if:

$$R_{ij} < P(d_{ij}) \tag{4}$$

3 Performance Optimizations

The function is designed for vectorized efficiency to support large-scale simulations:

Memory Efficiency The matrix is processed in sub-blocks based on m-type groups, reducing the peak memory footprint during probability assignment.

NumPy Masking By using boolean arrays (mask) and meshgrids (`np.ix_`), the implementation avoids iterating over individual cell pairs, allowing the construction of circuits with tens of thousands of neurons in seconds.

4 Usage Example

```
import numpy as np
from circuit_builder import build_adjacency_matrix

# Initialize parameters
coords = np.random.rand(1000, 3) * 500  # 1000 neurons in 500um cube
m_types = ['L23_PC'] * 1000
conn_data = load_markram_data()

# Generate Adjacency Matrix
adj_matrix = build_adjacency_matrix(coords, m_types, conn_data)
```